```
=> #<Project:0x007fcb918be958
 id: nil,
 name: "Project Runway",
 due_date: nil,
 created_at: nil,
 updated_at: nil,
 public: false>
```

Pry allows you to closely examine the current object under scope, which you can determine with the self command. Right now the current object is main:

```
[3] pry(main)> self
=> main
[4] pry(main)> ls
ActiveSupport::ToJsonWithActiveSupportEncoder#methods: to_json
Rails::ConsoleMethods#methods: app  controller  helper  new_session  reload!
self.methods: inspect  to_s
locals: _  __  _dir_  _ex_  _file_  _in_  _out_  _pry_  project
```

Pry uses the metaphor of Unix directory navigation to work through object trees (which, honestly, I find kind of weird). So, just as typing ls in a regular Unix terminal gives you a list of files, typing ls for Pry gives you a list of all kinds of stuff about the current namespace by introspecting on the object, including the available methods on the current object and local variables. (See the Pry wiki page on GitHub for full details.[7])

You can see that the list of locals contains a bunch of preexisting special variables and then the project variable that you created at the start of the sessions.

Just as you can type ls for a listing, you can type cd to change scope. Let's go into the scope of the project object:

```
[5] pry(main)> cd project
[6] pry(#<Project>):1> self
=> #<Project:0x007fcb918be958
 id: nil,
 name: "Project Runway",
 due_date: nil,
 created_at: nil,
 updated_at: nil,
 public: false>
[7] pry(#<Project>):1> ls
==> A BUNCH OF STUFF
```

Here the cd command is used to change scope. As with Unix, cd .. will take you back up a level, and other syntax quirks move you through multiple levels in a single command.

7. https://github.com/pry/pry/wiki/State-navigation